

The Agent Loop

Supplementary description of the delivery-robot experiment (Section 7.4, Table 4)

Contents

1	What the experiment asks	1
2	The scenario	2
2.1	Campus, rooms and robot	2
2.2	The planner and the two procedures	2
2.3	Perception	2
2.4	The two decisions	3
3	The builds	3
3.1	KRROOD: the objects are the knowledge base	3
3.2	GraphDB: a mirror in a server, the planner on the answers	5
3.3	GraphDB, push: the planner's results written into the store	6
3.4	Nemo: a Datalog engine re-run every step	8
3.5	reasonable (briefly)	9
4	How the rows of Table 4 are measured	9
4.1	Rows from the semantics	9
4.2	Rows from the implementation	10
4.3	The line count	11
5	Results	12

1 What the experiment asks

An agent that uses OWL knowledge rarely decides with logical inference alone. A delivery robot needs inferred facts (who is a member of a college, who is a T20 cricket fan) and it needs its own procedures (can it reach a room with its battery, how long does it take to drive there). These procedures read the robot's current state, which changes every step. They are *computables* in the sense of KnowRob: their values are computed when a query needs them, not stored.

The experiment asks two questions.

1. Can such an agent decide with *one* world model, in which perception updates the knowledge, the OWL 2 RL consequences follow, and the decision query calls the planner directly?
2. What does a *second* world model cost: in code that synchronizes the two representations, in statements written per step, in requests to another process, and in time per step?

To answer them, the same robot is built several ways. Everything except the knowledge layer is the same code in every build:

- **Scenario.** The campus map, the course rooms and mail boxes, the robot's start, and the perceptions of every step are generated once per seed from the raw OWL2Bench data, with RDFLib and without reasoning, and written to a file that every build reads (`scenario.py`). Seeds 0–4, 200 steps each.
- **Robot and planner.** The robot, its path planner (Dijkstra's algorithm on the map) and its two procedures `can_reach` and `travel_seconds` are one module (`robot.py`, `campus.py`).

- **Decisions.** The two decision queries ask the same question in every build (Section 2.4), and the policy that picks the action from their answers is one class, `DeliveryPolicy`.
- **States.** Every build except the reference (KRROOD) carries out the reference's action in every step, not its own. All builds therefore see the same robot states. Each build's own decision and its candidate sets are still recorded and compared with the reference's in every step (`loop.py`).

A build implements only four methods of `AgentVariant` (`loop.py`): `setup` (load the knowledge and the application data), `perceive` (bring the knowledge up to date with the step's facts), and `handout_candidates` and `ticket_candidates` (answer the two decisions). Each build runs in a fresh Python interpreter (`worker.py`), so builds share no memory, caches or symbol graph.

The paper's Table 4 compares four builds: KRROOD, GraphDB, GraphDB with pushed planner results, and Nemo. The paper's text also reports reasonable. A sixth build, `krrood_navigation`, is KRROOD with a different handout query (Section 3.1).

2 The scenario

2.1 Campus, rooms and robot

OWL2Bench has no rooms, so a campus map is generated from the university's organizations and a seed (`campus.py`). Every college is one building on a grid. A service hub in the middle holds the robot's main charging dock and the university mail room. Walkways connect neighboring buildings and the hub; each walkway is longer than the straight line by a seeded detour factor. Inside a building, a lobby leads to an elevator; every department occupies one floor, with a corridor and, on both sides, the department office, one laboratory and enough classrooms for the department's courses (five courses share a classroom). The first building has a second charging dock in its lobby. The map is a weighted, undirected graph. For seed 0 it has 379 places, 381 edges, 222 rooms and 2 docks.

Application data that the ontology lacks is attached by IRI:

- every course is taught in a classroom of the department that offers it (858 courses with a room, seed 0);
- every person's mail is delivered to the office of the person's department, or to the mail room if the person has none (2,616 people with a mail box, seed 0, including the people created by perception).

The robot starts at the hub dock with a full battery of 2,500 m of driving (an elevator ride counts as an equivalent length), drives at 1.2 m/s, and goes to charge below 25% of its capacity. Its pose and battery are plain attributes of the Robot object.

2.2 The planner and the two procedures

`PathPlanner` computes shortest distances with Dijkstra's algorithm. The distance field from a start place is computed once and kept, and the distance to the nearest dock is one multi-source run, because the robot plans from the same few places again and again. Two procedures sit on top of it:

```
def can_reach(robot: Robot, room: Place) -> bool:
    ..
    needed = robot.planner.distance(robot.place, room) + robot.planner.distance_to_nearest_dock(room)
    result = needed <= robot.battery_metres
    ..

def travel_seconds(robot: Robot, room: Place) -> float:
    ..
    result = robot.planner.distance(robot.place, room) / robot.specification.speed_metres_per_second
```

`can_reach` holds if the robot can drive to the room and from there to the nearest dock with its remaining battery. `travel_seconds` is the driving time. Both read the robot's state when called. Both record their calls and their time in a `ProcedureMeter`, which the step meter uses to book their time separately (Section 4).

2.3 Perception

Each step perceives three knowledge events and a requested college (`events.py`, `scenario.py`). Each event is one of three kinds, drawn independently with weights 0.25, 0.60 and 0.15:

- **Enrollment:** a new person enrolls in a department (person a `Person` ; `enrollIn department`). The person is new, so the functional `enrollIn` never gets a second value.

- **Course taking:** a student takes a course not taken yet, with probability 0.7 one of the student’s own department (student takesCourse course).
- **Cricket enthusiasm:** a person who is neither crazy about T20 cricket nor dislikes it becomes crazy about it (person isCrazyAbout T20Cricket). likes and dislikes are disjoint, so haters are excluded.

Over the 600 events of seed 0 there are 129 enrollments, 378 course takings and 93 cricket enthusiasms. Every event adds a new fact. No event removes one, because KRROOD does not retract facts inferred from a removed fact. Events name individuals by IRI, so the same input reaches every build. The requested college is drawn uniformly from the colleges.

2.4 The two decisions

Handouts. The students who are members of the requested college, with the courses they take whose room the robot can reach, and the driving time to that room. Membership is inferred. OWL2Bench states

$$\text{enrollIn} \circ \text{isPartOf} \sqsubseteq \text{isMemberOf}, \quad \text{isPartOf transitive},$$

and a department is part of its college. A person who just enrolled in a department is therefore a member of the department’s college only through the property chain. The domain of enrollIn makes the person a Student.

Tickets. The T20 cricket fans whose mail box the robot can reach, with the driving time. Fans are inferred from the class definition

$$\text{T20CricketFan} \equiv \exists \text{isCrazyAbout}.\{\text{T20Cricket}\}$$

(an OWL hasValue restriction). A person perceived as crazy about T20 cricket becomes a fan only through this axiom.

Action. DeliveryPolicy.decide(robot.py) is the same in every build:

- below the charge threshold, drive to the nearest dock and charge;
- otherwise drive to the nearest room with an undelivered item: a match ticket for a fan, or the handouts of the requested college for a course (one delivery per college and course). Ties go to the ticket, then to the smaller IRI;
- without such a room, charge if the battery is not full, and wait otherwise.

The policy remembers what it delivered; this is application state, not knowledge. Over seed 0 the reference delivers 96 tickets and 96 handouts and charges 8 times.

3 The builds

For every build we describe (a) how knowledge is represented, (b) how perception updates it, (c) how inference happens, (d) how the computables are called, and (e) how the decisions are queried. Code is quoted from the build’s file in src/krrood_experiments/aamas27/agent_loop/, shortened with “...”. The experiments use the earlier version of KRROOD that the supplement contains; names such as variable_from are those of that version.

How to read the diagrams. Each diagram shows one step from left to right. The top lane is the Python process of the agent. The bottom lane is the second process (the GraphDB server, the Nemo process) or, for reasonable, the native library inside the same process; for KRROOD it is empty. A node sits in the lane where its work runs. Red arrows cross a process boundary.  marks a request to a server (a round trip),  statements written to the second world model. Badges mark the code that the line count attributes (Section 4.3):  synchronization, which Table 4 reports,  mapping and  procedure integration, which are counted in the results but not reported in Table 4. Column headers name the phases of the step; a red header marks a phase that runs out of the usual order.

3.1 KRROOD: the objects are the knowledge base

(a) Representation. The knowledge base is the application’s own objects. Setup loads the raw OWL2Bench data with Ontomatic’s instance loader, exactly as the loading benchmark does, which creates the objects and materializes the closure (krrood_variant.py, setup). It then attaches the application data to the loaded objects: course.taught_in is a Place of the map and person.mailbox is a Place. The robot is an ordinary object next to them. There is one world model.

(b) Perception. Perceiving a fact is an assignment through a property descriptor. Events name individuals by IRI, so the build first resolves the IRIs to objects with the loader's registry; this is its only mapping code. A new person is a new Person object with a Student role.

```
def perceive(self, perception: StepPerception, meter: StepMeter) -> None:
    for event in perception.knowledge_events:
        if isinstance(event, Enrollment):
            with meter.phase("mapping"):
                department = self._resolve(event.department, model.Department)
            with meter.phase("update"):
                person = model.Person(uri=event.person)
                student = model.Student(person=person)
                person.mailbox = self.scenario.campus.place(self.scenario.mailbox_of(event.person))
                student.enroll_in.add(department)
            ...
        elif isinstance(event, CourseTaking):
            ...
            student.takes_course.add(course)
        elif isinstance(event, CricketEnthusiasm):
            ...
            person.is_crazy_about.add(self.t20_cricket)
```

(c) Inference. The descriptors forward-chain on assignment. EnrollIn is a sub-property of IsStudentOf; IsMemberOf declares the chain axioms (EnrollIn, IsPartOf) and (WorksFor, IsPartOf); IsPartOf is transitive. So student.enroll_in.add(department) adds the department's college (and the university) to student.is_member_of before the assignment returns. is_crazy_about.add also adds the value to loves and likes. This version of KRROOD creates role objects for defined classes only when loading. Membership in T20CricketFan is therefore derived on request: the query evaluates the class axiom of the generated model, which is

```
return (HasProperty(candidate_var, IsCrazyAbout),
        exists(to_str(candidate_var.is_crazy_about.uri) == "http://benchmark/OWL2Bench#T20Cricket"))
```

(d) Computables. The procedures are wrapped once, as a predicate and a symbolic function. These seven lines are the build's procedure-integration code.

```
@boundary(PROCEDURE_INTEGRATION)
@dataclass(eq=False)
class CanReach(Predicate):
    robot: Robot
    room: Place
    def __call__(self) -> bool:
        return can_reach(self.robot, self.room)

@boundary(PROCEDURE_INTEGRATION)
@symbolic_function
def travel_time(robot: Robot, room: Place) -> float:
    return travel_seconds(robot, room)
```

EQL evaluates them inside the query, for each binding that reaches them, on the live robot object. No value is stored.

(e) Decision queries. Both decisions are EQL queries over the live objects:

```
def handout_query(self, college: model.Organization):
    student = variable(model.Student, domain=None)
    course = variable_from(student.takes_course)
    driving_time = travel_time(self.robot, course.taught_in)
    query = an(set_of(student, course, driving_time).where(
        contains(student.is_member_of, college),
        CanReach(self.robot, course.taught_in)))
    ...

def ticket_query(self):
    person = variable(model.Person, domain=None)
    driving_time = travel_time(self.robot, person.mailbox)
    query = an(set_of(person, driving_time).where(
        *model.T20CricketFan.axiom(person),
        CanReach(self.robot, person.mailbox)))
```

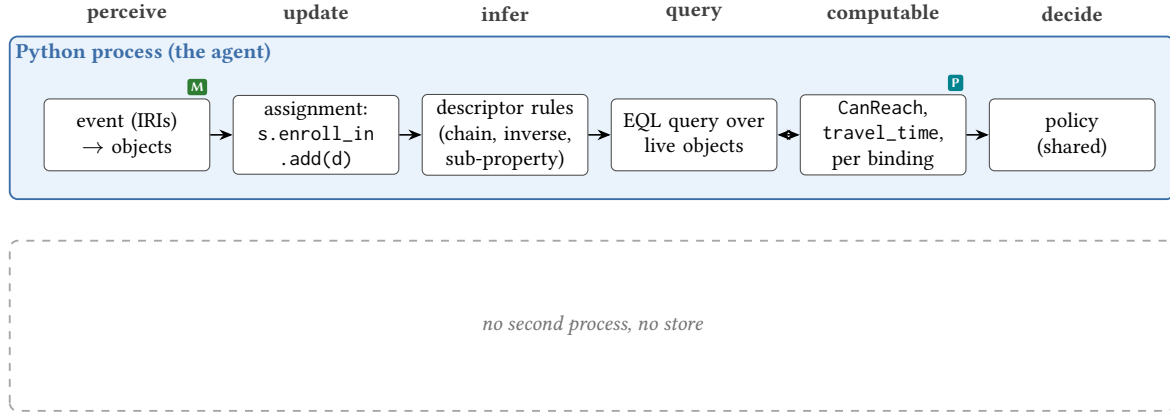


Figure 1: KRROOD, one step. Everything runs in the agent’s process on one world model: the inference happens in the assignment, and the planner is called from inside the query. 0 round trips, 0 statements written elsewhere.

The answers are objects; the build reads their IRIs and room identifiers to build the candidate records that the shared policy takes. `order_by` of this KRROOD version sorts only within one binding group, so the policy orders the candidates, as in every other build.

The navigation form. `krrood_navigation` (`krrood_navigation_variant.py`) changes only the handout query: it starts from the requested college, `variable(model.Student, domain=college.has_student)`, and drops the membership condition. `has_student` is the inverse of `isStudentOf` and is maintained by the descriptors; since `isPartOf` and `isSubOrganizationOf` are equivalent in OWL2Bench, the students of a college are exactly its student members. It is the “join order chosen by hand” of the paper’s text and makes the same decisions and candidate sets in every step.

3.2 GraphDB: a mirror in a server, the planner on the answers

(a) Representation. The application keeps its own objects: the robot, the campus map, and two lookups from IRI to Place for course rooms and mail boxes. The OWL knowledge is a second world model: a GraphDB 11.2 repository with the ruleset `owl2-rl-optimized`, into which setup loads the raw data; GraphDB materializes the closure during the load (`graphdb_variant.py`, class `GraphDBMirror`). The client keeps one HTTP connection open (`requests.Session`) and passes the requested college as a SPARQL binding, so the query texts stay constant (`rdf_mirror.py`, `GraphDBSession`).

(b) Perception. Every perceived fact is turned into RDF statements and sent in one `INSERT DATA` update per step. An enrollment is two statements, the other events one each, so a step writes 3 to 6 statements.

```
@boundary(SYNCHRONIZATION)
def event_triples(event: KnowledgeEvent) -> List[Tuple]:
    if isinstance(event, Enrollment):
        return [(event.person, RDF_TYPE, OWL2BENCH + "Person"),
                (event.person, OWL2BENCH + "enrollIn", event.department)]
    if isinstance(event, CourseTaking):
        return [(event.student, OWL2BENCH + "takesCourse", event.course)]
    if isinstance(event, CricketEnthusiasm):
        return [(event.person, OWL2BENCH + "isCrazyAbout", T20_CRICKET)]
    ...

@boundary(SYNCHRONIZATION)
def insert_data(triples: Iterable[Tuple[str, str, str]]) -> str:
    lines = [f"<{subject}> <{predicate}> {object_term(value)} ." for subject, predicate, value in triples]
    return "INSERT DATA {\n" + "\n".join(lines) + "\n}"

def perceive(self, perception: StepPerception, meter: StepMeter) -> None:
    with meter.phase("mapping"):
        triples = [triple for event in perception.knowledge_events for triple in event_triples(event)]
        update = insert_data(triples)
    with meter.phase("update"):
        self._send(update, meter)  # one HTTP request, counted as a round trip
    ...
```

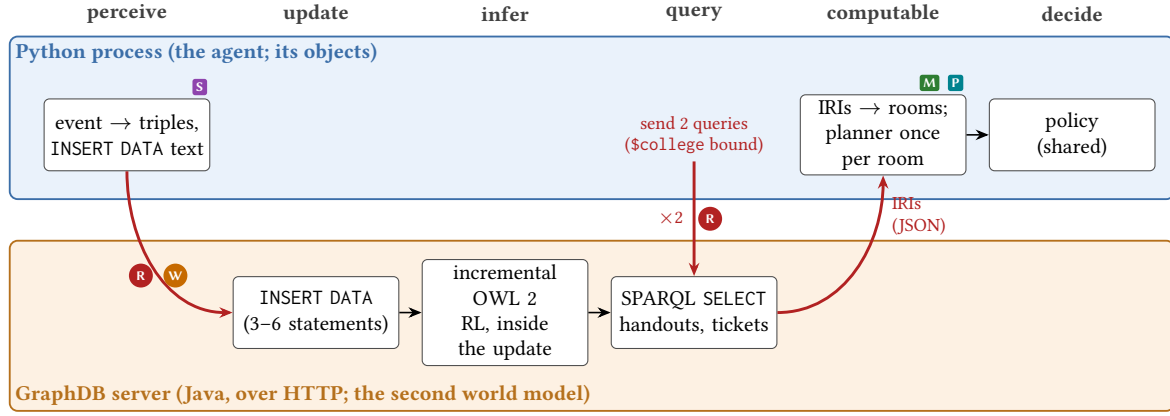


Figure 2: GraphDB, one step. Perception crosses into the server as an update (1 round trip, 3–6 statements), inference runs there, the two queries cross over and back (2 round trips), and the planner runs in Python on the answers after their IRIs are mapped back.

(c) Inference. GraphDB materializes the consequences of the inserted statements incrementally, inside the update’s transaction, before the request returns: Student from the domain of enrollIn, isMemberOf from the chain, T20CricketFan from the hasValue definition. The update’s time therefore includes the request, the commit and the inference.

(d) Computables. SPARQL cannot call the robot’s procedures. The query returns the IRIs that satisfy the logical part; the build maps them back to the application’s places and runs the planner in Python on the answers, once per distinct room:

```
@boundary(MAPPING)
def _handout_rows(self, solutions: List[Dict[str, str]]) -> List[Tuple[str, str, Place]]:
    return [(s["student"], s["course"], self.course_rooms[s["course"]]) for s in solutions]

@boundary(PROCEDURE_INTEGRATION)
def _reachable_driving_times(self, rooms: set) -> Dict[Place, float]:
    return {room: travel_seconds(self.robot, room) for room in rooms if can_reach(self.robot, room)}
```

(e) Decision queries. Two SPARQL queries per step, each one request (?college is bound per request):

```
SELECT ?student ?course WHERE {
  ?student a owlbench:Student ;
    owlbench:isMemberOf ?college ;
    owlbench:takesCourse ?course .
}
SELECT ?person WHERE {
  ?person a owlbench:T20CricketFan .
}
```

```
def handout_candidates(self, college: str, meter: StepMeter) -> List[HandoutCandidate]:
    with meter.phase("query"):
        solutions = self._select(HANDOUT_QUERY, meter, college=college)
    with meter.phase("mapping"):
        rows = self._handout_rows(solutions)
    with meter.phase("query"):
        reachable = self._reachable_driving_times({room for _, _, room in rows})
    return [HandoutCandidate(student, course, room.identifier, reachable[room])
            for student, course, room in rows if room in reachable]
```

A step thus makes three requests: one update and two queries.

3.3 GraphDB, push: the planner’s results written into the store

This build asks what it takes for SPARQL alone to decide (graphdb_push_variant.py). It shares the repository, the loading and the mirroring of facts with the GraphDB build.

(a) Representation. As in GraphDB, plus application data in the store under the prefix `agent:` (namespace `https://example.org/agent-loop#`): at setup, `agent:taughtIn` for every course and `agent:mailbox` for every person who exists at the start (`_application_triples`, 14 synchronization lines); every step, `agent:travelSeconds` for every room the robot can reach now.

(b) Perception. The first update of a step inserts the perceived facts and the mail box of each new person (`_knowledge_triples`). A second update replaces the planner's results:

```
def perceive(self, perception: StepPerception, meter: StepMeter) -> None:
    with meter.phase("mapping"):
        triples = self._knowledge_triples(perception)
        update = insert_data(triples)
    with meter.phase("update"):
        self._send(update, meter)
    ...
    with meter.phase("push"):
        values = self._procedure_results()
        self._send(DELETE_PROCEDURE_RESULTS + " ;\n" + insert_data(values), meter)
    meter.statements_deleted += self.pushed_statements
    meter.statements_inserted += len(values)
```

```
DELETE WHERE { ?room agent:travelSeconds ?seconds }
```

(c) Inference. As in GraphDB: incremental OWL 2 RL inside each update. The pushed driving times are plain data; no rule derives from them.

(d) Computables. The planner runs *before* the query, for every room a decision might need (the 201 course rooms and mail boxes of the scenario), because the store cannot ask for a value when it needs one. An unreachable room gets no value, which encodes `can_reach`:

```
@boundary(SYNCHRONIZATION)
def _procedure_results(self) -> List[Tuple]:
    return [(place_iri(room.identifier), AGENT_LOOP + "travelSeconds",
        f'"{travel_seconds(self.robot, room)!r}"^^<{DOUBLE}>')]
    for room in self.target_rooms if can_reach(self.robot, room)]
```

The robot moves almost every step, so nearly every value changes every step: about 200 statements are deleted and about 200 inserted.

(e) Decision queries. One SPARQL query per decision selects, filters by reachability (a join with the pushed values) and orders:

```
SELECT ?student ?course ?room ?seconds WHERE {
    ?student a owl2bench:Student ;
        owl2bench:isMemberOf ?college ;
        owl2bench:takesCourse ?course .
    ?course agent:taughtIn ?room .
    ?room agent:travelSeconds ?seconds .
}
ORDER BY ?seconds ?course ?student

SELECT ?person ?room ?seconds WHERE {
    ?person a owl2bench:T20CricketFan ;
        agent:mailbox ?room .
    ?room agent:travelSeconds ?seconds .
}
ORDER BY ?seconds ?person
```

The answers are converted into candidate records (`_handout_records`, `_ticket_records` and `place_identifier`, mapping code). No procedure-integration code is left; its work moved into synchronization code. A step makes four requests: two updates and two queries.

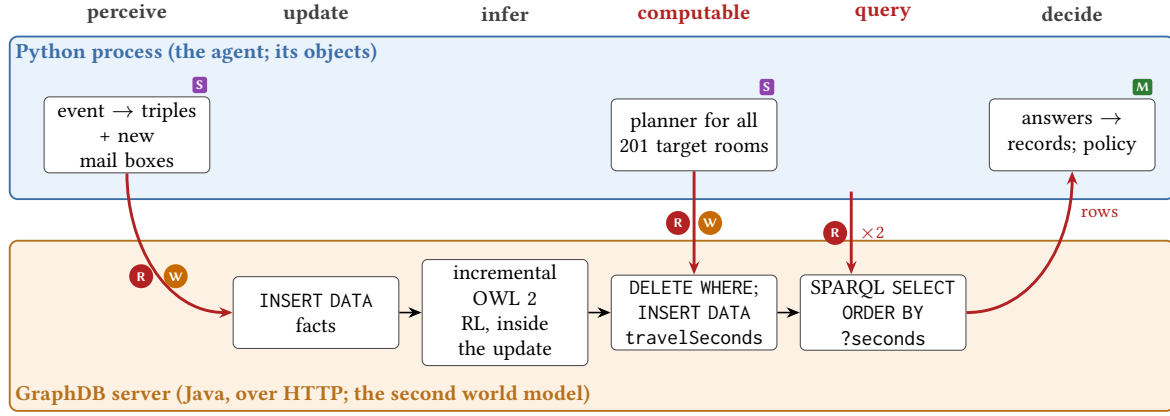


Figure 3: GraphDB with pushed planner results, one step. The planner runs before the query, for every room, and its results cross into the store with the facts (2 updates, about 200 statements deleted and 200 inserted); then SPARQL alone selects and orders (2 queries). 4 round trips.

3.4 Nemo: a Datalog engine re-run every step

Nemo 0.10.1 materializes a program in one batch run of its command-line tool `nmo` and has no incremental mode (`nemo_variant.py`).

(a) Representation. The application keeps its objects and IRI lookups, as in GraphDB. The OWL knowledge is the raw OWL2Bench data (RDF/XML) plus an N-Triples file `perceived.nt` that holds every fact perceived so far. The closure exists only during a run of `nmo`.

(b) Perception. The step's facts are appended to the file, with the same `event_triples` as the RDF builds:

```
@boundary(SYNCHRONIZATION)
def _append_facts(self, perception: StepPerception) -> int:
    lines = [f"<{subject}> <{predicate}> {object_term(value)} .\n"
             for event in perception.knowledge_events
             for subject, predicate, value in event_triples(event)]
    with open(self.work_directory / "perceived.nt", "a") as stream:
        stream.writelines(lines)
    return len(lines)
```

(c) Inference. Every step starts a new `nmo` process on one program: the OWL 2 RL/RDF rules (`owl2r1.rls`, written for Nemo) with their import and export declarations replaced, so that both the raw data and the perceived facts are input, and with the two decision rules added. Nemo computes the closure from scratch and exports only the two decision predicates as CSV. According to the build's documentation, restarting from the previous step's closure was slower in a trial, since Nemo treats every input fact as new.

```
@parameter $input = "input.rdf" .
@parameter $perceived = "perceived.nt" .
@parameter $college = <http://benchmark/OWL2Bench#College> .
@import raw :- rdf{resource = $input} .
@import perceived :- ntriples{resource = $perceived} .
input(?s, ?p, ?o) :- raw(?s, ?p, ?o) .
input(?s, ?p, ?o) :- perceived(?s, ?p, ?o) .
... % the OWL 2 RL/RDF rules
@export handout :- csv{resource = "handout.csv"} .
@export fan :- csv{resource = "fan.csv"} .
```

(d) Computables. Nemo's rules cannot call the robot's procedures. As in GraphDB, the exported IRIs are mapped to places and the planner runs in Python on the answers, once per distinct room (`_reachable_driving_times`, the same two lines).

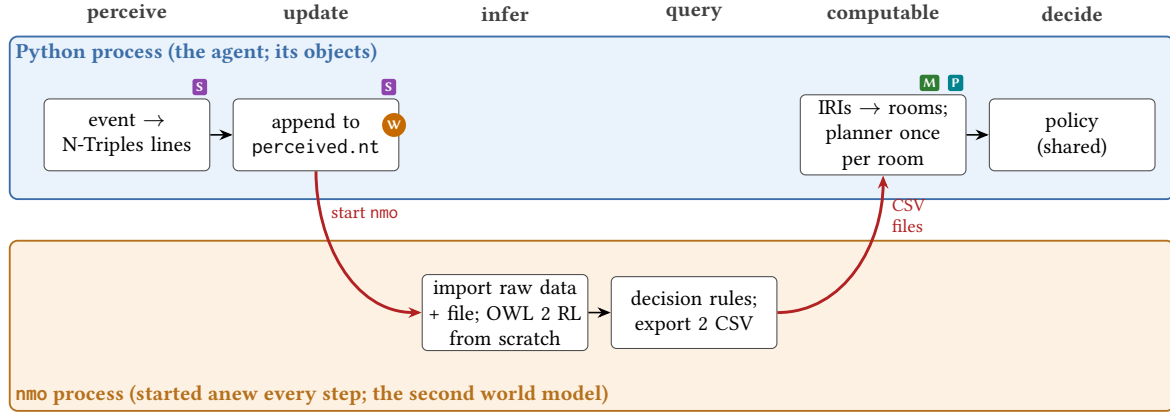


Figure 4: Nemo, one step. Perception appends to a file (about 4 statements); a new process recomputes the whole closure and the two decision rules; the planner runs in Python on the exported answers. No server request; one process start per step.

(e) Decision queries. The decisions are two rules over the closure predicate `triple/3` and its view `rdf_type/2`; the requested college is the parameter `$college`, passed on the command line:

```
handout(?student, ?course) :- rdf_type(?student, owl2bench:Student),
    triple(?student, owl2bench:isMemberOf, $college), triple(?student, owl2bench:takesCourse, ?course) .
fan(?person) :- rdf_type(?person, owl2bench:T20CricketFan) .
```

```
command = [self.nemo_binary, "--overwrite-results", "--export-dir", str(export_directory), "--report", "short",
    "--param", f'input="{Path(UNREASONED_FILE).resolve()}"',
    "--param", f'perceived="{self.work_directory / "perceived.nt"}"',
    "--param", f'college=<{college}>',
    str(self.work_directory / "step.rls")]
completed = subprocess.run(command, capture_output=True, text=True, check=True)
```

The run happens in perceive (phase “reasoning”); the two decision methods only read the stored answers. A step makes no server request; it starts one process and reads two files.

3.5 reasonable (briefly)

reasonable 0.4.4 is an in-memory OWL 2 RL reasoner written in Rust with Python bindings (`reasonable_variant.py`). Setup parses the raw data with `RDFLib` and materializes the closure. Every step adds the perceived facts as `RDFLib` statements with `from_graph` and calls `reason()`. `reasonable` materializes additions incrementally, but `reason()` returns the whole closure as a list of `RDFLib` triples, so the step includes building that list. The build then makes one pass over the returned triples (`ClosureIndex.of`) to collect students, fans, members and courses, maps the answers to places, and runs the planner on them, as `GraphDB` does. It runs in one process, but the closure is a second world model in another data structure, written in another language. `reasonable 0.4.4` does not implement the property-chain rule `prp-spo2`, so it never infers `isMemberOf` of a college and finds no handout candidates. This is recorded, not repaired. It was run for seed 0 only, as it needs about 17 GB.

4 How the rows of Table 4 are measured

Table 4 groups its rows by what they follow from. The upper rows follow from evaluating EQL queries over the program’s objects: a build of this kind has one world model, calls the planner in the query and writes nothing to another store. The lower rows follow from the implementation: one language and one process could also come from a wrapper that generates SPARQL or from an embedded store, which would still keep two world models.

4.1 Rows from the semantics

World models. The number of representations of the robot’s world knowledge that the build keeps up to date: the objects (always), plus the `GraphDB` repository (both `GraphDB` builds), the input files and Nemo’s closure (Nemo), or `reasonable`’s closure. Read from the code.

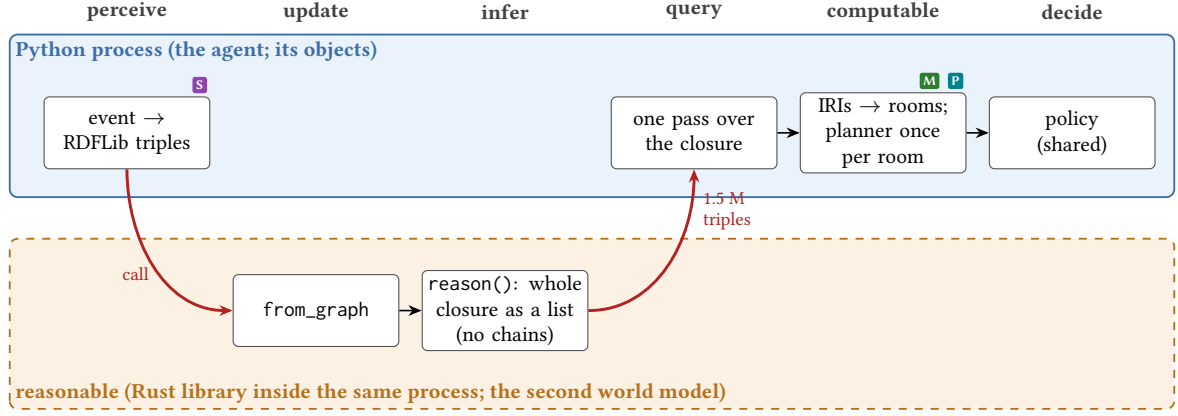


Figure 5: reasonable, one step. Two calls into the native library (no server request); the whole closure comes back every step and is scanned in Python.

Planner runs. Where `can_reach` and `travel_seconds` are evaluated relative to the logical query. *In query:* as an EQL predicate and symbolic function, per binding, on the live robot. *On answers:* in Python, after the query, once per distinct room among the answers (GraphDB, Nemo, reasonable). *Before query:* for every room a decision could need, with the results written into the store (GraphDB push). The ProcedureMeter counts the calls. For seed 0, the mean number of `can_reach` calls per step is 1,310.7 for KRROOD (per binding), 59.5 for GraphDB (per distinct room among the answers) and 201 for GraphDB push (every target room) (`agent_loop.json`).

Writes per step. Statements written to the second world model in a step, inserted plus deleted, as counted by the build in `StepMeter.statements_inserted` and `statements_deleted`: the statements of the `INSERT DATA` updates (GraphDB), plus the deleted and inserted `travelSeconds` values (push), or the lines appended to `perceived.nt` (Nemo). The median over the 1,000 steps of the five seeds is reported. KRROOD has no second world model, so the table reports 0. Its meter does count its descriptor assignments in `statements_inserted` (median 5 per step: three per enrollment, counting the new person and the mail box, and one per other event), but these are the update itself, not a copy of it.

Synchronization lines. Lines of code that exist only to write the application’s facts into the second representation, counted as in Section 4.3.

4.2 Rows from the implementation

Languages. The languages in which the developer writes the decision: Python (KRROOD); Python and SPARQL (both GraphDB builds); Python and Nemo’s rule language (Nemo).

Processes. The operating-system processes that hold or compute the robot’s knowledge: the Python worker (KRROOD, reasonable); the worker and the GraphDB server, a Java process reached over HTTP (GraphDB builds); the worker and one `nmo` process started per step (Nemo).

Round trips per step. Requests to a server in a step. `GraphDBMirror._send` (updates) and `_select` (queries) increment `StepMeter.round_trips` once per HTTP request. The count is constant: 3 for GraphDB (one update, two queries) and 4 for push (two updates, two queries). KRROOD, Nemo and reasonable make no server requests; Nemo’s process start and reasonable’s two library calls per step are counted separately as `reasoner_calls` (1 and 2).

Step [ms]. The wall time of a step, from `StepMeter` (`measurement.py`). The loop (`loop.py`, `run_agent_loop`) times every part of a step as a phase with `time.perf_counter`:

- mapping: translating between the objects and another representation;
- update: writing the perceived facts, including the inference that happens on insertion (descriptor chaining in KRROOD, GraphDB’s incremental materialization inside the update request);
- reasoning: a separate materialization call (Nemo’s `run`, reasonable’s `reason()`);

Table 1: The marked functions and their code lines, from the `boundary_lines` entries of the results files.

KRROOD (0 / 12 / 7)			GraphDB (16 / 9 / 2)		
<code>_attach_application_data</code>	M	8	<code>event_triples</code>	S	11
<code>_resolve</code>	M	2	<code>insert_data</code>	S	3
<code>_register</code>	M	2	<code>object_term</code>	S	2
<code>CanReach</code>	P	5	<code>_application_data</code>	M	4
<code>travel_time</code>	P	2	<code>_handout_rows, _ticket_rows</code>	M	2, 3
			<code>_reachable_driving_times</code>	P	2
GraphDB, push (44 / 15 / 0)			Nemo (22 / 9 / 2)		
<code>_application_triples</code>	S	14	<code>_append_facts</code>	S	9
<code>_knowledge_triples</code>	S	8	<code>event_triples</code>	S	11
<code>_procedure_results</code>	S	6	<code>object_term</code>	S	2
<code>event_triples, insert_data, object_term</code>	S	11, 3, 2	<code>_application_data</code>	M	4
<code>_application_data</code>	M	4	<code>_handout_rows, _ticket_rows</code>	M	2, 3
<code>_handout_records, _ticket_records</code>	M	5, 2	<code>_reachable_driving_times</code>	P	2
<code>place_iri, place_identifier</code>	M	2, 2			

- push: writing the planner’s results into the store (push only);
- query: evaluating the decision queries, without the time inside the procedures;
- procedures: the time inside `can_reach` and `travel_seconds`, wherever they are called from. The ProcedureMeter times every call, and the enclosing phase’s time is reduced by it;
- decide and act: the shared policy and the robot’s motion.

The step time is the sum of the phases. It excludes setup (loading and materializing the closure) and the bookkeeping of the experiment (the digest of the candidates). The table reports the median over the 1,000 steps of the five seeds and, in parentheses, the range of the five seeds’ medians; the results file also holds a 95% bootstrap interval of the median and the 95th percentile (`agent_loop_seeds.json`).

Agreement. Not a row, but the precondition for comparing the rows. For every step, the build’s own action and its candidate sets (sorted, de-duplicated, driving times rounded to microseconds, hashed) are compared with the reference’s.

4.3 The line count

The line count. Code is attributed by marking functions with the decorator `@boundary(category)` of `measurement.py`. There are three categories:

- SYNCHRONIZATION: code that writes the application’s facts into a second representation;
- MAPPING: code that translates between the application’s objects and another representation (IRIs to objects, objects to IRIs, query results to application records);
- PROCEDURE_INTEGRATION: code that makes the robot’s procedures usable in a decision, either a predicate class or symbolic function that wraps a procedure, or the Python filter applied to the answers of a query that cannot call it.

`boundary_functions` collects the marked methods of the build’s class and its base classes (the most derived definition of each name) and the module-level functions the class lists in `boundary_helpers` (for example the shared `event_triples`, `insert_data` and `object_term` of `rdf_mirror.py`). `code_lines` counts the lines of each function’s source that hold code: it drops blank lines, comment lines, decorator lines and every string-literal statement (function and attribute docstrings). The `def` or `class` line counts. Table 1 lists the counted functions. The bodies of `perceive`, `handout_candidates` and `ticket_candidates` are not marked in any build; they are the step’s control flow, which every build has.

Mapping and procedure lines: counted, not in Table 4. The line count also attributes lines of code that translate between the application’s objects and another representation (mapping) and lines that make the robot’s procedures usable in a decision (procedure integration). The results files hold them (`boundary_lines`, per category; Table 1), and the supplement’s report lists them, but the paper’s Table 4 does not report them, as they do not measure a second world model. The mapping lines follow from the simulated perceptions, which arrive as IRIs in every build: KRROOD, with one world model, also resolves them to objects (12 lines). The procedure lines follow from wrapping the planner, as a predicate and a function in KRROOD (7 lines) or as a filter of the answers in GraphDB and Nemo (2 lines), not from the number of world models.

5 Results

Runs. The numbers of Table 4 come from the measured run of the supplement’s results (results/agent_loop/): seeds 0–4, 200 steps each, three perceived facts per step, one run per build and seed, on an Intel Core i7-13700 with 64 GB RAM under Ubuntu 24.04, in a Docker container pinned to six cores, with GraphDB 11.2 Free (one core, 6 GB heap) in the same container (provenance.txt). Nemo was run separately, five seeds of 200 steps on other cores, with KRROOD alongside as the reference of the agreement check.

The table. The paper’s Table 4 reads:

	KRROOD	GraphDB	GraphDB, push	Nemo
<i>From the semantics</i>				
World models	1	2	2	2
Planner runs	in query	on answers	before query	on answers
Writes / step	0	4	405	4
Synchronization lines	0	16	44	22
<i>From the implementation</i>				
Languages	1	2	2	2
Processes	1	2	2	2
Round trips / step	0	3	4	1 run
Step [ms]	90 (88–90)	96 (93–100)	162 (150–166)	5,683 (5,653–5,690)

GraphDB push writes 405 statements per step: 204 inserted and 201 deleted (medians).

What the rows show.

- *One world model is enough.* KRROOD decides with no synchronization code, no writes to another store and no requests. Outside the table, its 12 mapping lines resolve the IRIs in which events name individuals and attach the application data, and its 7 procedure lines are the predicate and the function (Section 4.3).
- *A second world model costs code and traffic.* GraphDB needs 16 lines to mirror facts and three requests per step; outside the table, 9 more lines map answers back and 2 filter them with the planner.
- *Letting the store decide moves the planner out of the decision.* With the planner’s results pushed, SPARQL decides alone and the procedure lines drop to 0, but the synchronization lines grow to 44, the writes to about 400 statements per step, and the planner runs for every room in every step, whether a decision needs it or not.
- *Step times are comparable, but spent differently.* The median phases (agent_loop_seeds.json) are: KRROOD, 88.3 ms in queries and 0.43 ms in the update; GraphDB, 88.9 ms in the update and 7.5 ms in queries; push, 87.2 ms in the update, 63.0 ms in the push and 15.5 ms in queries. A GraphDB update costs about 38 ms even when empty (median of 30 empty updates, graphdb_update_cost.json). The navigation form of KRROOD’s handout query brings KRROOD’s median step to 74 ms.
- *Recomputing the closure every step is slow.* Nemo makes the same decisions with a median step of 5.68 s (seeds’ medians 5.65–5.69 s), nearly all of it Nemo’s run (by its own report, about 2.2 s import and 3.4 s reasoning). In the table, Nemo has 2 world models, runs the planner on the answers, appends a median of 4 statements per step, has 22 synchronization lines, uses 2 languages and 2 processes, and makes no server request but starts one process per step (“1 run”); outside the table, it has 9 mapping and 2 procedure lines. reasonable takes 4.6 s per step (seed 0) and finds no handout candidate.

Agreement. Both GraphDB builds and Nemo make KRROOD’s decision and return KRROOD’s candidate sets in all 1,000 steps. For Nemo the candidate files of both are identical in every seed. reasonable agrees on the action in 104 of 200 steps and on the candidate sets in 2, because it derives no property chains.